

Learning in an Echo Chamber

Amartya Sanyal

Assistant Professor of Machine Learning, University of Copenhagen
Adjunct Assistant Professor, IIT Kanpur

Foundations of Responsible Machine Learning
<https://cope-forml.github.io/>

Based on joint work with Daniil Dmitriev, Carolin Heinzler, and Harald Eskelund Franck

Two replay papers from our group

Today's talk focuses on the first.

TALK FOCUS

“Learning in an Echo Chamber: Online Learning with Replay Adversary”

arXiv:2509.25135

Accepted at **SODA 2026**



Daniil
Dmitriev



Harald
Eskelund
Franck



Carolin
Heinzler

ALSO ON ARXIV



Giorgio
Racca



Michal
Valko

“Language Generation with Replay: A Learning-Theoretic View of Model Collapse”

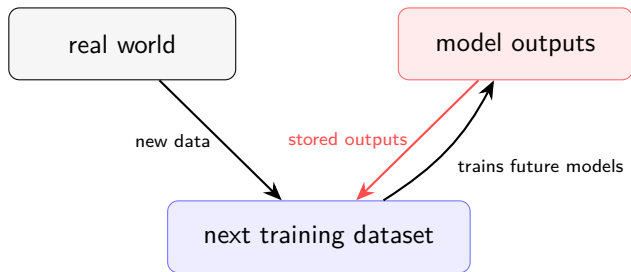
arXiv:2603.11784

Not today's topic.

Motivation: what is the problem?

Modern models do not just read the world. They also write data that future models may train on.

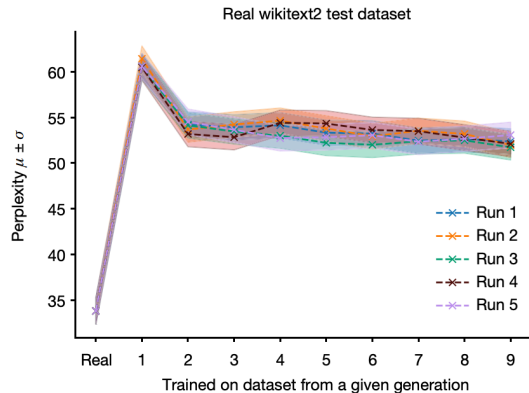
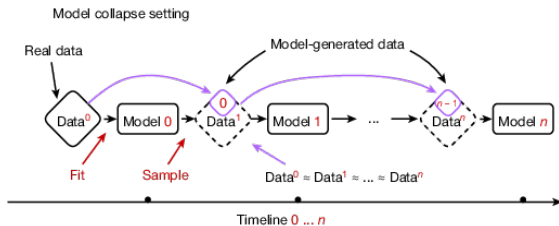
- Labels, rankings, and generated text can persist.
- Later training rounds may treat them as fresh observations.



Model Collapse in LLMs

Recent work on model collapse in LLMs shows that recursive training can distort the learned distribution.

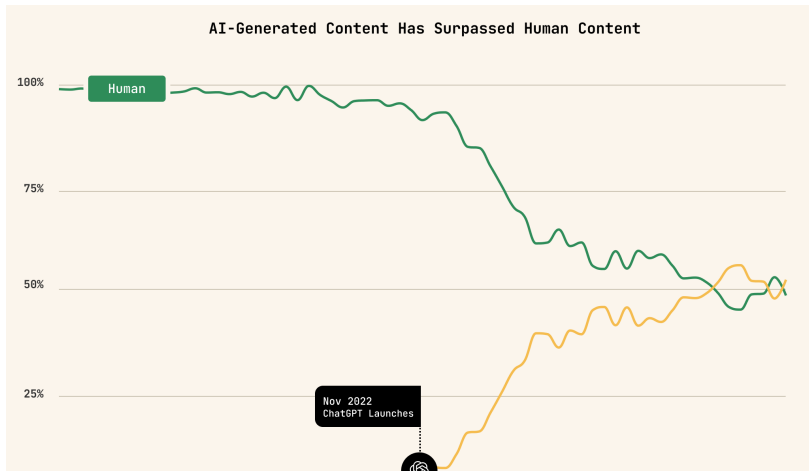
Shumailov et al., *Nature* 631 (2024), Fig. 1 screenshot.



Machine-generated content is growing

If more of the web is machine-written, future training corpora contain model output by default.

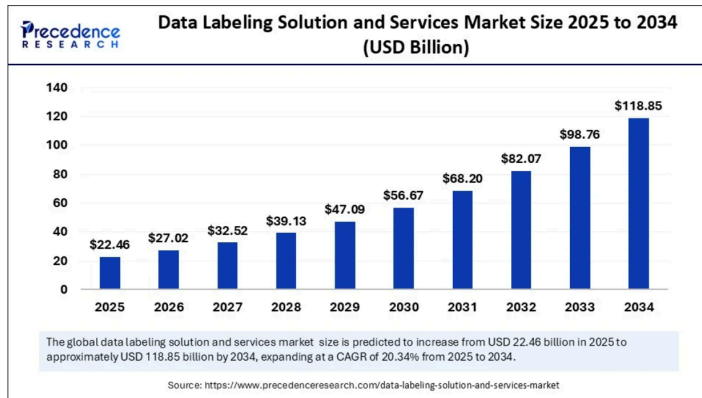
Context graphic from Graphite.



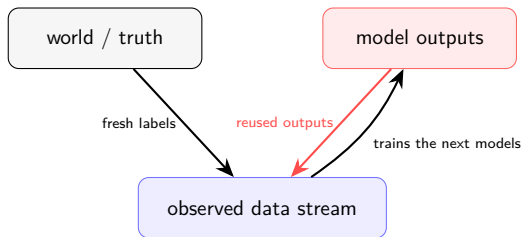
Data Annotated is a big industry

Fresh human labels are expensive at scale, so many pipelines reuse, transform, or weakly supervise labels instead of recollecting truth from scratch.

Context graphic from Precedence Research.



Replay in learning



Replay is the common structure in all of these loops.

- Fresh truth and old outputs both feed the next dataset.
- The learner sees the label, but not its source.

Replay is not ordinary label noise: it depends on the learner's own past behavior.

Classical online learning vs. replay

Classical online learning

- **Feedback:** revealed label is true.
- **Can every mistake be trusted?** Yes.
- **What can go wrong?** The sequence may be hard, but the feedback is reliable.

Classical online learning vs. replay

Classical online learning

- **Feedback:** revealed label is true.
- **Can every mistake be trusted?** Yes.
- **What can go wrong?** The sequence may be hard, but the feedback is reliable.

Replay

- **Feedback:** the revealed label may be truth or a past output.
- **Can every mistake be trusted?** No.
- **What can go wrong?** Ambiguous feedback can reinforce the learner's own history.

From classical online learning to replay

Classical online learning game

- 1 Nature fixes a target hypothesis $h^* \in \mathcal{H}$.
- 2 On round t , nature presents x_t .
- 3 Learner predicts $\hat{y}_t$.
- 4 Revealed label is the true one:
$$y_t = h^*(x_t).$$
- 5 Learner updates and continues.

From classical online learning to replay

Classical online learning game

- 1 Nature fixes a target hypothesis $h^* \in \mathcal{H}$.
- 2 On round t , nature presents x_t .
- 3 Learner predicts $\hat{y}_t$.
- 4 Revealed label is the true one:

$$y_t = h^*(x_t).$$

- 5 Learner updates and continues.

Replay changes the label step

We only change one line in the classical game:

$$y_t \in \{h^*(x_t)\} \cup \{\hat{y}_s(x_t) : s < t\}.$$

From classical online learning to replay

Classical online learning game

- 1 Nature fixes a target hypothesis $h^* \in \mathcal{H}$.
- 2 On round t , nature presents x_t .
- 3 Learner predicts $\hat{y}_t$.
- 4 Learner observes a label:

$$y_t \in \{h^*(x_t)\} \cup \{\hat{y}_s(x_t) : s < t\}$$

- 5 Learner updates and continues.

From classical online learning to replay

Classical online learning game

- 1 Nature fixes a target hypothesis $h^* \in \mathcal{H}$.
- 2 On round t , nature presents x_t .
- 3 Learner predicts $\hat{y}_t$.
- 4 Learner observes a label:

$$y_t \in \{h^*(x_t)\} \cup \{\hat{y}_s(x_t) : s < t\}$$

- 5 Learner updates and continues.

How mistakes are counted

We count only mistakes on true labels:

$$\text{mistake}_t = \mathbf{1}\{\hat{y}_t \neq y_t\} \mathbf{1}\{y_t = h^*(x_t)\}.$$

The issue is that the learner sees y_t , but does not know whether y_t came from truth or replay.

Two messages to keep in mind

Adaptive replay has an exact worst-case answer. We show a new dimension, ExThD, that characterizes it.

Two messages to keep in mind

Adaptive replay has an exact worst-case answer. We show a new dimension, ExThD, that characterizes it.

- Under stochastic replay, random covariates often make the problem milder.

Two messages to keep in mind

Adaptive replay has an exact worst-case answer. We show a new dimension, ExThD, that characterizes it.

- Under stochastic replay, random covariates often make the problem milder.
- We first see this on thresholds, then lift the same idea to general classes.

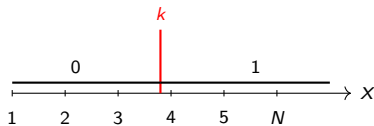
Warm-up: thresholds

The simplest class already shows the replay phenomenon.

Warm-up class: one-dimensional thresholds

Domain $\mathcal{X} = [N]$. Threshold class:

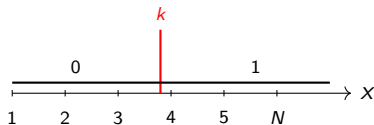
$$\mathcal{H}_{\text{thresh}} = \{f_k : \mathcal{X} \rightarrow \{0, 1\} \mid k \in \{0, 1, \dots, N\}\},$$
$$f_k(x) = \mathbf{1}\{x \geq k\} \quad (\text{so } f_0 \equiv 0).$$



Warm-up class: one-dimensional thresholds

Domain $\mathcal{X} = [N]$. Threshold class:

$$\mathcal{H}_{\text{thresh}} = \{f_k : \mathcal{X} \rightarrow \{0, 1\} \mid k \in \{0, 1, \dots, N\}\},$$
$$f_k(x) = \mathbf{1}\{x \geq k\} \quad (\text{so } f_0 \equiv 0).$$



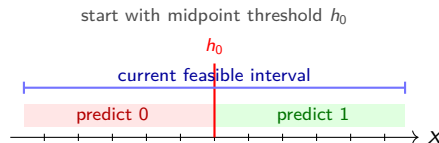
- In the classical mistake-bound model: $O(\log N)$ mistakes.

Classical threshold strategy: binary search

Algorithm

Maintain a feasible interval $[L_t, R_t]$ for the unknown threshold.

- Predict with the midpoint threshold $h_t(x) = \mathbf{1}\{x \geq k_t\}$, where k_t is the center of $[L_t, R_t]$.

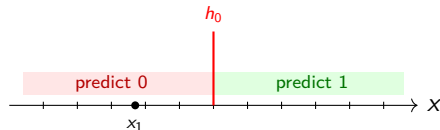


Classical threshold strategy: binary search

Algorithm

Maintain a feasible interval $[L_t, R_t]$ for the unknown threshold.

- Predict with the midpoint threshold $h_t(x) = \mathbf{1}\{x \geq k_t\}$, where k_t is the center of $[L_t, R_t]$.
- The next point x_t arrives.

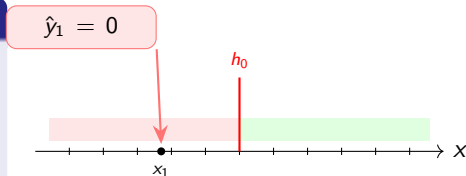


Classical threshold strategy: binary search

Algorithm

Maintain a feasible interval $[L_t, R_t]$ for the unknown threshold.

- Predict with the midpoint threshold $h_t(x) = \mathbf{1}\{x \geq k_t\}$, where k_t is the center of $[L_t, R_t]$.
- The next point x_t arrives.

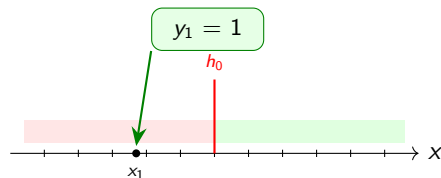


Classical threshold strategy: binary search

Algorithm

Maintain a feasible interval $[L_t, R_t]$ for the unknown threshold.

- Predict with the midpoint threshold $h_t(x) = \mathbf{1}\{x \geq k_t\}$, where k_t is the center of $[L_t, R_t]$.
- The next point x_t arrives.
- If $\hat{y}_t \neq y_t$, the mistake tells us which side of x_t the true threshold lies on.

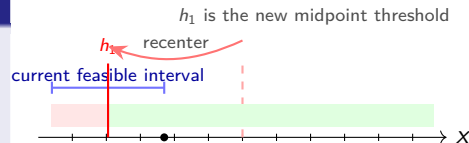


Classical threshold strategy: binary search

Algorithm

Maintain a feasible interval $[L_t, R_t]$ for the unknown threshold.

- Predict with the midpoint threshold $h_t(x) = \mathbf{1}\{x \geq k_t\}$, where k_t is the center of $[L_t, R_t]$.
- The next point x_t arrives.
- If $\hat{y}_t \neq y_t$, the mistake tells us which side of x_t the true threshold lies on.
- Keep that surviving interval and recenter there.

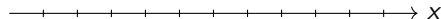


Classical threshold strategy: binary search

Algorithm

Maintain a feasible interval $[L_t, R_t]$ for the unknown threshold.

- Predict with the midpoint threshold $h_t(x) = \mathbf{1}\{x \geq k_t\}$, where k_t is the center of $[L_t, R_t]$.
- The next point x_t arrives.
- If $\hat{y}_t \neq y_t$, the mistake tells us which side of x_t the true threshold lies on.
- Keep that surviving interval and recenter there.

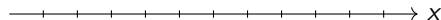


Classical threshold strategy: binary search

Algorithm

Maintain a feasible interval $[L_t, R_t]$ for the unknown threshold.

- Predict with the midpoint threshold $h_t(x) = \mathbf{1}\{x \geq k_t\}$, where k_t is the center of $[L_t, R_t]$.
- The next point x_t arrives.
- If $\hat{y}_t \neq y_t$, the mistake tells us which side of x_t the true threshold lies on.
- Keep that surviving interval and recenter there.



each mistake removes at least half of the candidate thresholds

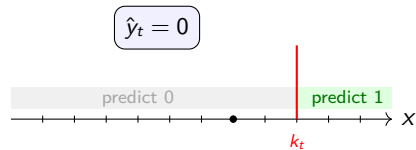
Each true mistake halves the feasible interval, so the classical mistake bound is $O(\log N)$.

Conservative threshold strategy

Algorithm

Start with $\hat{h}_1 \equiv 0$. For each round t :

- Predict $\hat{y}_t = \hat{h}_t(x_t)$.

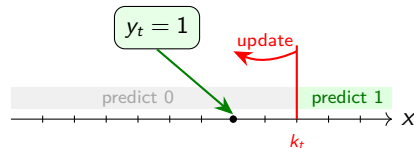


Conservative threshold strategy

Algorithm

Start with $\hat{h}_1 \equiv 0$. For each round t :

- Predict $\hat{y}_t = \hat{h}_t(x_t)$.
- If $y_t = 1$ and $\hat{y}_t = 0$, then **update** $\hat{h}_{t+1} := \mathbf{1}\{\cdot \geq x_t\}$.

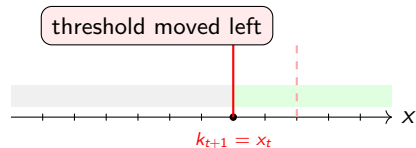


Conservative threshold strategy

Algorithm

Start with $\hat{h}_1 \equiv 0$. For each round t :

- Predict $\hat{y}_t = \hat{h}_t(x_t)$.
- If $y_t = 1$ and $\hat{y}_t = 0$, then **update** $\hat{h}_{t+1} := \mathbf{1}\{\cdot \geq x_t\}$.
- Otherwise, do not update: $\hat{h}_{t+1} := \hat{h}_t$.

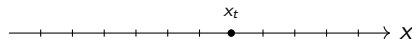


Conservative threshold strategy

Algorithm

Start with $\hat{h}_1 \equiv 0$. For each round t :

- Predict $\hat{y}_t = \hat{h}_t(x_t)$.
- If $y_t = 1$ and $\hat{y}_t = 0$, then **update** $\hat{h}_{t+1} := \mathbf{1}\{\cdot \geq x_t\}$.
- Otherwise, do not update: $\hat{h}_{t+1} := \hat{h}_t$.



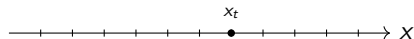
Unlike the classical case, a positive label is no longer enough to rule out half the thresholds: it may be replay.

Conservative threshold strategy

Algorithm

Start with $\hat{h}_1 \equiv 0$. For each round t :

- Predict $\hat{y}_t = \hat{h}_t(x_t)$.
- If $y_t = 1$ and $\hat{y}_t = 0$, then **update** $\hat{h}_{t+1} := \mathbf{1}\{\cdot \geq x_t\}$.
- Otherwise, do not update: $\hat{h}_{t+1} := \hat{h}_t$.



Unlike the classical case, a positive label is no longer enough to rule out half the thresholds: it may be replay.

Theorem

Against any adaptive replay adversary on $\mathcal{H}_{\text{thresh}}$ over $[M]$, the conservative strategy makes

$$M_T = O(\min\{N, T\}) \quad \text{mistakes.}$$

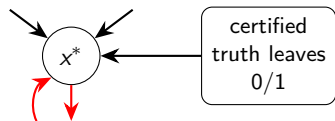
Any learner can be forced to make (lower bound) $M_T = \Omega(\min\{N, T\})$ mistakes.

Trap region and $\Omega(T)$ lower bound intuition

Definition (informal)

A point (or region) x is **trapped** if:

- **Certified truth does not resolve it:** based on the certified labels so far, both $f^*(x) = 0$ and $f^*(x) = 1$ are possible.

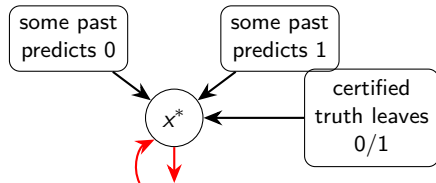


Trap region and $\Omega(\mathcal{T})$ lower bound intuition

Definition (informal)

A point (or region) x is **trapped** if:

- **Certified truth does not resolve it:** based on the certified labels so far, both $f^*(x) = 0$ and $f^*(x) = 1$ are possible.
- **Replayable disagreement exists:** among the learner's past hypotheses, some predict 0 on x and some predict 1 on x .



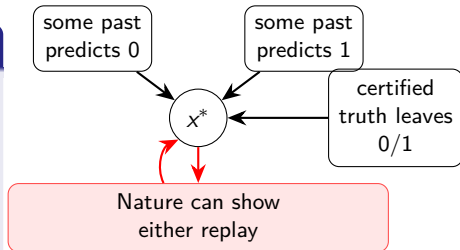
Trap region and $\Omega(T)$ lower bound intuition

Definition (informal)

A point (or region) x is **trapped** if:

- **Certified truth does not resolve it:** based on the certified labels so far, both $f^*(x) = 0$ and $f^*(x) = 1$ are possible.
- **Replayable disagreement exists:** among the learner's past hypotheses, some predict 0 on x and some predict 1 on x .

Why this matters: nature can now show *either* a replayed 0 or a replayed 1 at x forever.



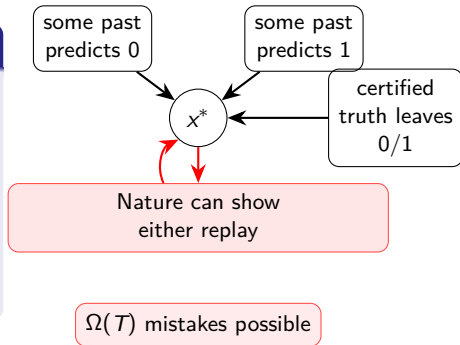
Trap region and $\Omega(\mathcal{T})$ lower bound intuition

Definition (informal)

A point (or region) x is **trapped** if:

- **Certified truth does not resolve it:** based on the certified labels so far, both $f^*(x) = 0$ and $f^*(x) = 1$ are possible.
- **Replayable disagreement exists:** among the learner's past hypotheses, some predict 0 on x and some predict 1 on x .

Why this matters: nature can now show *either* a replayed 0 or a replayed 1 at x forever.



Proof intuition for thresholds

Upper bound (conservative strategy)

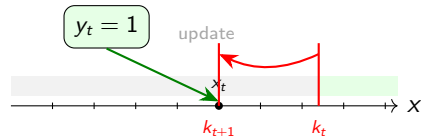
- Let k_t be the threshold at time t , so $\hat{h}_t(x) = \mathbf{1}\{x \geq k_t\}$.



Proof intuition for thresholds

Upper bound (conservative strategy)

- Let k_t be the threshold at time t , so $\hat{h}_t(x) = \mathbf{1}\{x \geq k_t\}$.
- Every certified mistake forces $k_{t+1} < k_t$, so $M_T \leq \min\{N, T\}$.



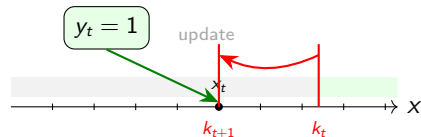
Proof intuition for thresholds

Upper bound (conservative strategy)

- Let k_t be the threshold at time t , so $\hat{h}_t(x) = \mathbf{1}\{x \geq k_t\}$.
- Every certified mistake forces $k_{t+1} < k_t$, so $M_T \leq \min\{N, T\}$.

Lower bound (adversary)

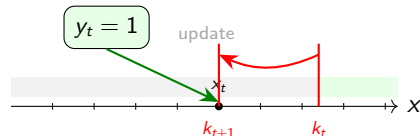
- Either there is a **trap**, giving $\Omega(T)$, or there is **no trap**, giving $\Omega(\min\{N, T\})$.



Proof intuition for thresholds

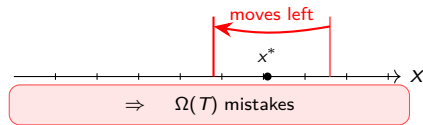
Upper bound (conservative strategy)

- Let k_t be the threshold at time t , so $\hat{h}_t(x) = \mathbf{1}\{x \geq k_t\}$.
- Every certified mistake forces $k_{t+1} < k_t$, so $M_T \leq \min\{N, T\}$.



Lower bound (adversary)

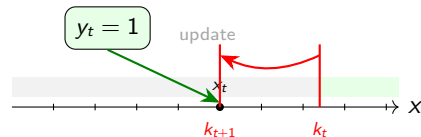
- Either there is a **trap**, giving $\Omega(T)$, or there is **no trap**, giving $\Omega(\min\{N, T\})$.
- **Trap case:** query x^* forever and replay the opposite past label.



Proof intuition for thresholds

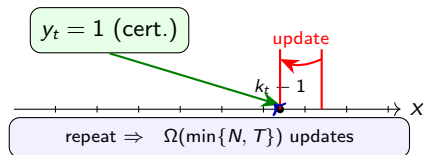
Upper bound (conservative strategy)

- Let k_t be the threshold at time t , so $\hat{h}_t(x) = \mathbf{1}\{x \geq k_t\}$.
- Every certified mistake forces $k_{t+1} < k_t$, so $M_T \leq \min\{N, T\}$.



Lower bound (adversary)

- Either there is a **trap**, giving $\Omega(T)$, or there is **no trap**, giving $\Omega(\min\{N, T\})$.
- **Trap case:** query x^* forever and replay the opposite past label.
- **No-trap case:** query $x_t = k_t - 1$ and reveal $y_t = 1$ (cert.) to force updates.

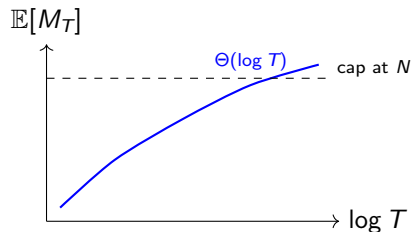


Thresholds under stochastic replay: $\Theta(\min\{N, \log T\})$

Theorem

For $x_t \sim D$ i.i.d. (stochastic adversary), the conservative strategy satisfies

$$\mathbb{E}[M_T] = \Theta(\min\{N, \log T\}).$$



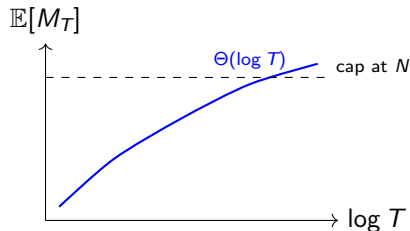
Thresholds under stochastic replay: $\Theta(\min\{N, \log T\})$

Theorem

For $x_t \sim D$ i.i.d. (stochastic adversary), the conservative strategy satisfies

$$\mathbb{E}[M_T] = \Theta(\min\{N, \log T\}).$$

- Intuition: mistakes correspond to **record minima** among reliable positive samples.
- An i.i.d. sequence has only $\Theta(\log T)$ record minima in expectation.



Takeaway

Under randomness, the adversary can no longer keep forcing the same trap forever; for thresholds, the replay cost drops from linear to logarithmic.

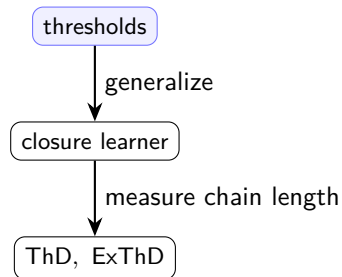
Now the same idea for general classes

Keep only certified truth, and grow conservatively.

What survives from thresholds

Lessons from the warm-up

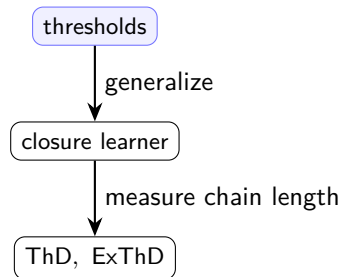
- Replay becomes dangerous when the learner creates a **trap**: a point where both labels remain plausible and both appear in the learner's history.



What survives from thresholds

Lessons from the warm-up

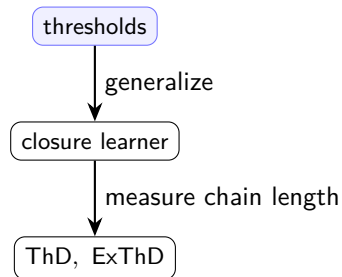
- Replay becomes dangerous when the learner creates a **trap**: a point where both labels remain plausible and both appear in the learner's history.
- Good algorithms are **conservative**: they enlarge the positive region only when a label is certified to be true.



What survives from thresholds

Lessons from the warm-up

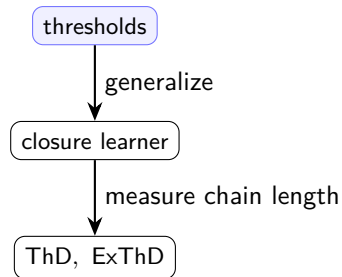
- Replay becomes dangerous when the learner creates a **trap**: a point where both labels remain plausible and both appear in the learner's history.
- Good algorithms are **conservative**: they enlarge the positive region only when a label is certified to be true.
- Then the mistake bound is controlled by the **length of a monotone update chain**.



What survives from thresholds

Lessons from the warm-up

- Replay becomes dangerous when the learner creates a **trap**: a point where both labels remain plausible and both appear in the learner's history.
- Good algorithms are **conservative**: they enlarge the positive region only when a label is certified to be true.
- Then the mistake bound is controlled by the **length of a monotone update chain**.



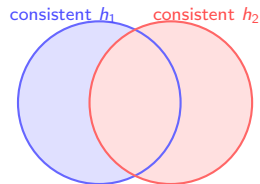
Takeaway

The general theory keeps the same three ingredients: certification, conservative updates, and chain-length control.

The general conservative learner

Closure learner (the threshold idea in general form)

- Keep only **certified positives**: revealed $y_t = 1$ that cannot be replay.

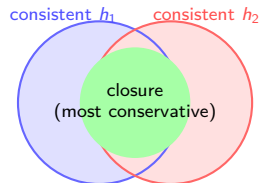


The general conservative learner

Closure learner (the threshold idea in general form)

- Keep only **certified positives**: revealed $y_t = 1$ that cannot be replay.
- Predict the most conservative consistent hypothesis:

$$\hat{h}_t = \text{clos}_{\mathcal{H}}(S_t) := \bigcap_{h \in \mathcal{H}: S_t \subseteq h} h.$$



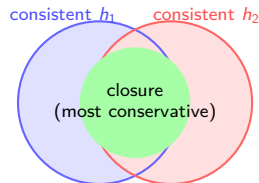
The general conservative learner

Closure learner (the threshold idea in general form)

- Keep only **certified positives**: revealed $y_t = 1$ that cannot be replay.
- Predict the most conservative consistent hypothesis:

$$\hat{h}_t = \text{clos}_{\mathcal{H}}(S_t) := \bigcap_{h \in \mathcal{H}: S_t \subseteq h} h.$$

- If a new certified positive lies outside $\hat{h}_t$, update $S_{t+1} := S_t \cup \{x_t\}$.



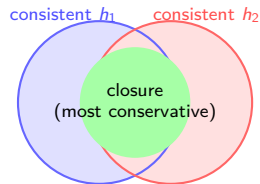
The general conservative learner

Closure learner (the threshold idea in general form)

- Keep only **certified positives**: revealed $y_t = 1$ that cannot be replay.
- Predict the most conservative consistent hypothesis:

$$\hat{h}_t = \text{clos}_{\mathcal{H}}(S_t) := \bigcap_{h \in \mathcal{H}: S_t \subseteq h} h.$$

- If a new certified positive lies outside $\hat{h}_t$, update $S_{t+1} := S_t \cup \{x_t\}$.



predict 1 only where
all consistent hypotheses agree

Threshold analogy

For thresholds, you moved a cutoff left. Closure does the same thing abstractly: enlarge the predicted positive region only when truth forces it.

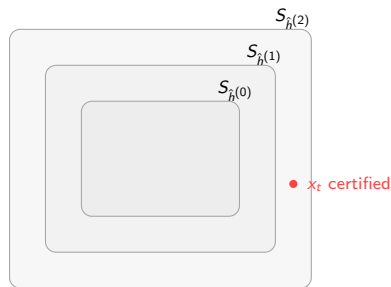
Why certified mistakes form a chain

Why “ordered by inclusion”?

Think of each hypothesis h as its **positive region**

$$S_h := \{x \in \mathcal{X} : h(x) = 1\} \subseteq \mathcal{X}.$$

Then $h \preceq h'$ means $S_h \subseteq S_{h'}$: h' predicts 1 on *at least* the points where h predicts 1, so it is less conservative.



Why certified mistakes form a chain

Why “ordered by inclusion”?

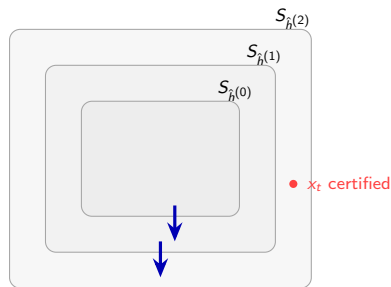
Think of each hypothesis h as its **positive region**

$$S_h := \{x \in \mathcal{X} : h(x) = 1\} \subseteq \mathcal{X}.$$

Then $h \preceq h'$ means $S_h \subseteq S_{h'}$: h' predicts 1 on *at least* the points where h predicts 1, so it is less conservative.

Why this gives an upper bound

- Prediction is $\hat{h}_t = \text{clos}_{\mathcal{H}}(S_t)$: the **smallest** positive region consistent with the certified positives S_t .



Why certified mistakes form a chain

Why “ordered by inclusion”?

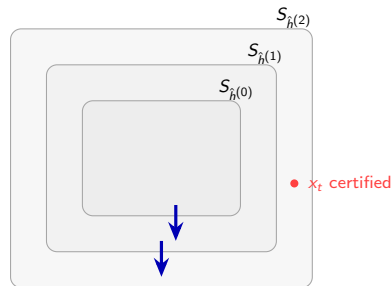
Think of each hypothesis h as its **positive region**

$$S_h := \{x \in \mathcal{X} : h(x) = 1\} \subseteq \mathcal{X}.$$

Then $h \preceq h'$ means $S_h \subseteq S_{h'}$: h' predicts 1 on *at least* the points where h predicts 1, so it is less conservative.

Why this gives an upper bound

- Prediction is $\hat{h}_t = \text{clos}_{\mathcal{H}}(S_t)$: the **smallest** positive region consistent with the certified positives S_t .
- A certified mistake gives $x_t \notin S_{\hat{h}_t}$ but $x_t \in S_{f^*}$, so we add it to S_t and the closure must **strictly grow**: $S_{\hat{h}_t} \subsetneq S_{\hat{h}_{t+1}}$.



Why certified mistakes form a chain

Why “ordered by inclusion”?

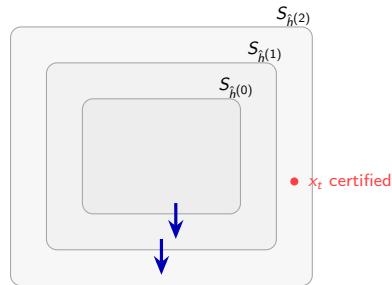
Think of each hypothesis h as its **positive region**

$$S_h := \{x \in \mathcal{X} : h(x) = 1\} \subseteq \mathcal{X}.$$

Then $h \preceq h'$ means $S_h \subseteq S_{h'}$: h' predicts 1 on *at least* the points where h predicts 1, so it is less conservative.

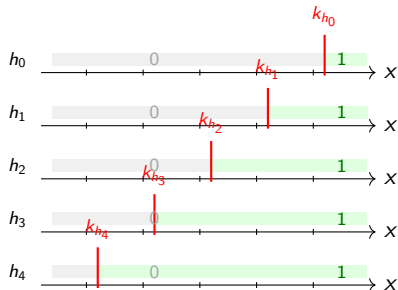
Why this gives an upper bound

- Prediction is $\hat{h}_t = \text{clos}_{\mathcal{H}}(S_t)$: the **smallest** positive region consistent with the certified positives S_t .
- A certified mistake gives $x_t \notin S_{\hat{h}_t}$ but $x_t \in S_{f^*}$, so we add it to S_t and the closure must **strictly grow**: $S_{\hat{h}_t} \subsetneq S_{\hat{h}_{t+1}}$.
- So the updates form a **strict chain** under $\subseteq$. Mistakes are at most the maximum chain length in the closure family.



From thresholds to a staircase matrix

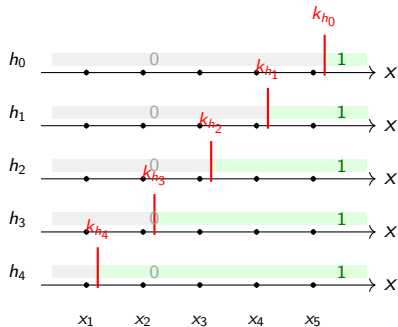
As thresholds become less conservative, the positive region grows one point at a time. In the label matrix, that appears as a staircase.



	x_1	x_2	x_3	x_4	x_5
h_0					
h_1					
h_2					
h_3					
h_4					

From thresholds to a staircase matrix

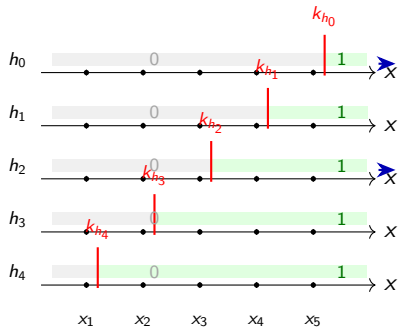
As thresholds become less conservative, the positive region grows one point at a time. In the label matrix, that appears as a staircase.



	x_1	x_2	x_3	x_4	x_5
h_0	0	0	0	0	0
h_1	0	0	0	0	1
h_2	0	0	0	1	1
h_3	0	0	1	1	1
h_4	0	1	1	1	1

From thresholds to a staircase matrix

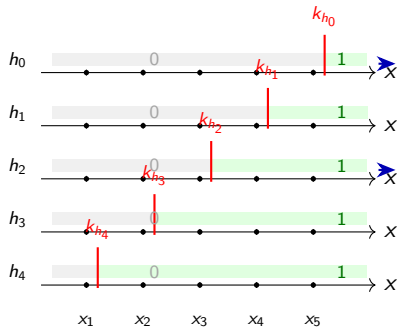
As thresholds become less conservative, the positive region grows one point at a time. In the label matrix, that appears as a staircase.



	x_1	x_2	x_3	x_4	x_5
h_0	0	0	0	0	0
h_1	0	0	0	0	1
h_2	0	0	0	1	1
h_3	0	0	1	1	1
h_4	0	1	1	1	1

From thresholds to a staircase matrix

As thresholds become less conservative, the positive region grows one point at a time. In the label matrix, that appears as a staircase.



	x_1	x_2	x_3	x_4	x_5
h_0	0	0	0	0	0
h_1	0	0	0	0	1
h_2	0	0	0	1	1
h_3	0	0	1	1	1
h_4	0	1	1	1	1

The staircase pattern measured by $\text{ThD}(\mathcal{H})$

Staircase witness

After ordering the chosen rows and columns, the witness has

$$h_i(x_j) = \mathbf{1}\{j \leq i\}.$$

Each new row gains one more 1.

	x_1	x_2	x_3	x_4	x_5
h_0	0	0	0	0	0
h_1	1	0	0	0	0
h_2	1	1	0	0	0
h_3	1	1	1	0	0
h_4	1	1	1	1	0
h_5	1	1	1	1	1

The staircase pattern measured by $\text{ThD}(\mathcal{H})$

Staircase witness

After ordering the chosen rows and columns, the witness has

$$h_i(x_j) = \mathbf{1}\{j \leq i\}.$$

Each new row gains one more 1.

$\text{ThD}(\mathcal{H})$ is the largest k for which $\mathcal{H}$ contains such a witness on k points and $k+1$ hypotheses.

	x_1	x_2	x_3	x_4	x_5
h_0	0	0	0	0	0
h_1	1	0	0	0	0
h_2	1	1	0	0	0
h_3	1	1	1	0	0
h_4	1	1	1	1	0
h_5	1	1	1	1	1

The staircase pattern measured by $\text{ThD}(\mathcal{H})$

Staircase witness

After ordering the chosen rows and columns, the witness has

$$h_i(x_j) = \mathbf{1}\{j \leq i\}.$$

Each new row gains one more 1.

$\text{ThD}(\mathcal{H})$ is the largest k for which $\mathcal{H}$ contains such a witness on k points and $k+1$ hypotheses.

For thresholds, this is exactly the one-by-one growth we just saw.

	x_1	x_2	x_3	x_4	x_5
h_0	0	0	0	0	0
h_1	1	0	0	0	0
h_2	1	1	0	0	0
h_3	1	1	1	0	0
h_4	1	1	1	1	0
h_5	1	1	1	1	1

A new complexity notion

Extended threshold dimension

Relabel, take closure, then measure $\text{ExThD}(\mathcal{H})$

Choose a baseline f , rewrite the class relative to it, complete it by intersections, then reveal the staircase.

1. Start with $\mathcal{H}$

	x_1	x_2	x_3	x_4
h_1	1	1	0	1
h_2	0	1	0	1
h_3	0	1	1	0
f				

2. Form $\mathcal{H}^f$ and close it

3. Reorder to see the staircase

three hypotheses in the original class

Relabel, take closure, then measure $\text{ExThD}(\mathcal{H})$

Choose a baseline f , rewrite the class relative to it, complete it by intersections, then reveal the staircase.

1. Start with $\mathcal{H}$

	x_1	x_2	x_3	x_4
h_1	1	1	0	1
h_2	0	1	0	1
h_3	0	1	1	0
f	0	0	1	1

2. Form $\mathcal{H}^f$ and close it

3. Reorder to see the staircase

now choose the baseline vector f

Relabel, take closure, then measure $\text{ExThD}(\mathcal{H})$

Choose a baseline f , rewrite the class relative to it, complete it by intersections, then reveal the staircase.

1. Start with $\mathcal{H}$

	x_1	x_2	x_3	x_4
h_1	1	1	0	1
h_2	0	1	0	1
h_3	0	1	1	0
f	0	0	1	1

now choose the baseline vector f

2. Form $\mathcal{H}^f$ and close it

	x_1	x_2	x_3	x_4
h_1^f	1	1	1	0
h_2^f	0	1	1	0
h_3^f	0	1	0	1
$h_2^f \cap h_3^f$				
$\overline{\mathcal{H}^f}$				

relabel by f : this is the new class $\mathcal{H}^f$

3. Reorder to see the staircase

Relabel, take closure, then measure $\text{ExThD}(\mathcal{H})$

Choose a baseline f , rewrite the class relative to it, complete it by intersections, then reveal the staircase.

1. Start with $\mathcal{H}$

	x_1	x_2	x_3	x_4
h_1	1	1	0	1
h_2	0	1	0	1
h_3	0	1	1	0
f	0	0	1	1

now choose the baseline vector f

2. Form $\mathcal{H}^f$ and close it

	x_1	x_2	x_3	x_4
h_1^f	1	1	1	0
h_2^f	0	1	1	0
h_3^f	0	1	0	1
h_4^f				

missing row:
 $h_2^f \cap h_3^f = (0, 1, 0, 0)$
so $\mathcal{H}^f$ is not
intersection-closed

here is the intersection row that is still
missing

3. Reorder to see the staircase

Relabel, take closure, then measure $\text{ExThD}(\mathcal{H})$

Choose a baseline f , rewrite the class relative to it, complete it by intersections, then reveal the staircase.

1. Start with $\mathcal{H}$

	x_1	x_2	x_3	x_4
h_1	1	1	0	1
h_2	0	1	0	1
h_3	0	1	1	0
f	0	0	1	1

2. Form $\mathcal{H}^f$ and close it

	x_1	x_2	x_3	x_4
h_1^f	1	1	1	0
h_2^f	0	1	1	0
h_3^f	0	1	0	1
$h_2^f \cap h_3^f$	0	1	0	0
$\overline{\mathcal{H}^f}$				

3. Reorder to see the staircase

now choose the baseline vector f

closure adds that missing row, giving $\overline{\mathcal{H}^f}$

Relabel, take closure, then measure $\text{ExThD}(\mathcal{H})$

Choose a baseline f , rewrite the class relative to it, complete it by intersections, then reveal the staircase.

1. Start with $\mathcal{H}$

	x_1	x_2	x_3	x_4
h_1	1	1	0	1
h_2	0	1	0	1
h_3	0	1	1	0
f	0	0	1	1

now choose the baseline vector f

2. Form $\mathcal{H}^f$ and close it

	x_1	x_2	x_3	x_4
h_1^f	1	1	1	0
h_2^f	0	1	1	0
h_3^f	0	1	0	1
$h_2^f \cap h_3^f$	0	1	0	0
$\overline{\mathcal{H}^f}$				

closure adds that missing row, giving $\overline{\mathcal{H}^f}$

3. Reorder to see the staircase

	x_2	x_3	x_1	x_4
g_0	1	0	0	0
g_1	1	1	0	0
g_2	1	1	1	0

reorder rows and columns; now the staircase is explicit

Relabel, take closure, then measure $\text{ExThD}(\mathcal{H})$

Choose a baseline f , rewrite the class relative to it, complete it by intersections, then reveal the staircase.

1. Start with $\mathcal{H}$

	x_1	x_2	x_3	x_4
h_1	1	1	0	1
h_2	0	1	0	1
h_3	0	1	1	0
f	0	0	1	1

now choose the baseline vector f

2. Form $\mathcal{H}^f$ and close it

	x_1	x_2	x_3	x_4
h_1^f	1	1	1	0
h_2^f	0	1	1	0
h_3^f	0	1	0	1
$h_2^f \cap h_3^f$	0	1	0	0
$\overline{\mathcal{H}^f}$				

closure adds that missing row, giving $\overline{\mathcal{H}^f}$

3. Reorder to see the staircase

	x_2	x_3	x_1	x_4
g_0	1	0	0	0
g_1	1	1	0	0
g_2	1	1	1	0

reorder rows and columns; now the staircase is explicit

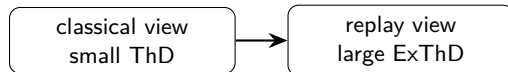
$$\text{ExThD}(\mathcal{H}) = \min_{f \subseteq \mathcal{X}} \text{ThD}(\overline{\mathcal{H}^f})$$

How ThD and ExThD compare

What always holds

$$\text{ThD}(\mathcal{H})/2 \leq \text{ExThD}(\mathcal{H}) \leq \text{ThD}(\overline{\mathcal{H}}) \leq |\mathcal{H}|.$$

$\overline{\mathcal{H}}$ is the intersection closure where the conservative learner moves.



How ThD and ExThD compare

What always holds

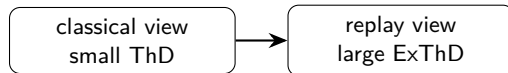
$$\text{ThD}(\mathcal{H})/2 \leq \text{ExThD}(\mathcal{H}) \leq \text{ThD}(\overline{\mathcal{H}}) \leq |\mathcal{H}|.$$

$\overline{\mathcal{H}}$ is the intersection closure where the conservative learner moves.

Memorable separation

There are classes with

$$\text{ThD}(\mathcal{H}) = 3 \quad \text{but} \quad \text{ExThD}(\mathcal{H}) \geq N.$$



How ThD and ExThD compare

What always holds

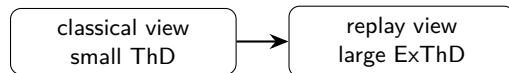
$$\text{ThD}(\mathcal{H})/2 \leq \text{ExThD}(\mathcal{H}) \leq \text{ThD}(\overline{\mathcal{H}}) \leq |\mathcal{H}|.$$

$\overline{\mathcal{H}}$ is the intersection closure where the conservative learner moves.

Memorable separation

There are classes with

$$\text{ThD}(\mathcal{H}) = 3 \quad \text{but} \quad \text{ExThD}(\mathcal{H}) \geq N.$$



Takeaway

ExThD is the right complexity measure for replay.

This gives the exact adaptive replay answer

The right quantity is $\text{ExThD}(\mathcal{H})$.

Main theorem: ExThD exactly captures adaptive replay

Theorem (paper, Thm. 3)

There is a (possibly improper) closure-based learner such that for every hypothesis class $\mathcal{H}$,

$$M_T \leq \text{ExThD}(\mathcal{H}) \quad (\text{adaptive replay})$$

Conversely, for $T \geq \text{ExThD}(\mathcal{H})$, any learner can be forced to make

$$M_T = \Omega(\text{ExThD}(\mathcal{H})).$$

Main theorem: ExThD exactly captures adaptive replay

Theorem (paper, Thm. 3)

There is a (possibly improper) closure-based learner such that for every hypothesis class $\mathcal{H}$,

$$M_T \leq \text{ExThD}(\mathcal{H}) \quad (\text{adaptive replay})$$

Conversely, for $T \geq \text{ExThD}(\mathcal{H})$, any learner can be forced to make

$$M_T = \Omega(\text{ExThD}(\mathcal{H})).$$

Interpretation

$\text{ExThD}(\mathcal{H})$ is the *exact* analogue of Littlestone dimension for replay.

For fixed f , Algorithm 1 achieves $M_T \leq \text{ThD}(\mathcal{H}^f)$ efficiently; minimizing over f can be exponential-time.

Why this theorem is true

1. Upper bound

Pick the best relabeling and run the conservative closure learner.

Why this theorem is true

1. Upper bound

Pick the best relabeling and run the conservative closure learner.

2. Trap alternative

If the learner ever creates a trap, replay can loop there forever.

Why this theorem is true

1. Upper bound

Pick the best relabeling and run the conservative closure learner.

2. Trap alternative

If the learner ever creates a trap, replay can loop there forever.

3. No-trap alternative

If traps are avoided, the learner must move monotonically through a long staircase chain.

Why this theorem is true

1. Upper bound

Pick the best relabeling and run the conservative closure learner.

2. Trap alternative

If the learner ever creates a trap, replay can loop there forever.

3. No-trap alternative

If traps are avoided, the learner must move monotonically through a long staircase chain.

certified-only closure updates



trap created
 $\Rightarrow$ replay loop



no traps
 $\Rightarrow$ forced chain

What changes under stochastic replay

Stochastic adversary: $x_t \sim D$ i.i.d. for a fixed distribution D over $\mathcal{X}$.

Transition from the adaptive case

The adversary no longer gets to keep querying the worst point forever. Random covariates weaken its ability to maintain a replay loop.

- Replays still exist, but they arrive on random covariates rather than chosen trap points.

What changes under stochastic replay

Stochastic adversary: $x_t \sim D$ i.i.d. for a fixed distribution D over $\mathcal{X}$.

Transition from the adaptive case

The adversary no longer gets to keep querying the worst point forever. Random covariates weaken its ability to maintain a replay loop.

- Replays still exist, but they arrive on random covariates rather than chosen trap points.
- For intersection-closed classes, the closure algorithm stays consistent on certified samples.

What changes under stochastic replay

Stochastic adversary: $x_t \sim D$ i.i.d. for a fixed distribution D over $\mathcal{X}$.

Transition from the adaptive case

The adversary no longer gets to keep querying the worst point forever. Random covariates weaken its ability to maintain a replay loop.

- Replays still exist, but they arrive on random covariates rather than chosen trap points.
- For intersection-closed classes, the closure algorithm stays consistent on certified samples.
- Sample-complexity control then becomes a bound on the *expected number of forced mistakes*.

What we can prove under stochastic replay

Theorem

Let $\text{VCdim}(\mathcal{H}) = d_{\text{vc}}$.

What we can prove under stochastic replay

Theorem

Let $\text{VCdim}(\mathcal{H}) = d_{\text{vc}}$.

- If $\mathcal{H}$ is intersection-closed, the closure algorithm gives

$$\mathbb{E}[M_T] = O(\min\{\text{ThD}(\mathcal{H}), d_{\text{vc}} \log T\}).$$

What we can prove under stochastic replay

Theorem

Let $\text{VCdim}(\mathcal{H}) = d_{\text{vc}}$.

- If $\mathcal{H}$ is intersection-closed, the closure algorithm gives

$$\mathbb{E}[M_T] = O(\min\{\text{ThD}(\mathcal{H}), d_{\text{vc}} \log T\}).$$

- For any $\mathcal{H}$, there exists a distribution D and target $f^* \in \mathcal{H}$ such that any deterministic learner has

$$\mathbb{E}[M_T] = \Omega(\min\{\text{ExThD}(\mathcal{H}), \log T\}).$$

What we can prove under stochastic replay

Theorem

Let $\text{VCdim}(\mathcal{H}) = d_{\text{vc}}$.

- If $\mathcal{H}$ is intersection-closed, the closure algorithm gives

$$\mathbb{E}[M_T] = O(\min\{\text{ThD}(\mathcal{H}), d_{\text{vc}} \log T\}).$$

- For any $\mathcal{H}$, there exists a distribution D and target $f^* \in \mathcal{H}$ such that any deterministic learner has

$$\mathbb{E}[M_T] = \Omega(\min\{\text{ExThD}(\mathcal{H}), \log T\}).$$

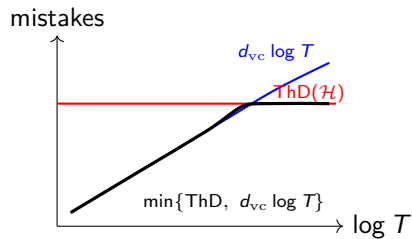
Setting	Upper bound	Lower bound
Adaptive replay	$\text{ExThD}(\mathcal{H})$	$\text{ExThD}(\mathcal{H})$
Stochastic replay (int.-closed)	$\min\{\text{ThD}(\mathcal{H}), d_{\text{vc}} \log T\}$	$\min\{\text{ExThD}(\mathcal{H}), \log T\}$

Upper bound uses closure +

PAC-style guarantees on reliable samples; the general stochastic characterization remains open.

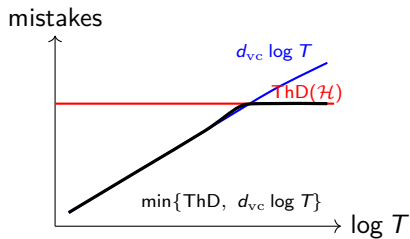
Why $d_{\text{vc}} \log T$?

- Think of $\hat{h}_t$ as trained on certified i.i.d. positives.



Why $d_{\text{vc}} \log T$?

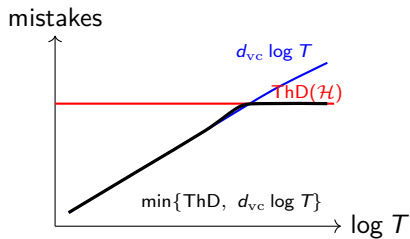
- Think of $\hat{h}_t$ as trained on certified i.i.d. positives.
- For intersection-closed classes, after m reliable samples, the generalization error is about $O(d_{\text{vc}}/m)$.



Why $d_{\text{vc}} \log T$?

- Think of $\hat{h}_t$ as trained on certified i.i.d. positives.
- For intersection-closed classes, after m reliable samples, the generalization error is about $O(d_{\text{vc}}/m)$.
- Summing those mistake probabilities gives a harmonic series:

$$\sum_{t=1}^T O\left(\frac{d_{\text{vc}}}{t}\right) = O(d_{\text{vc}} \log T).$$

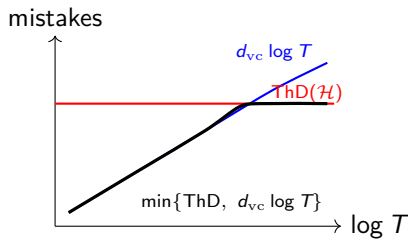


Why $d_{\text{vc}} \log T$?

- Think of $\hat{h}_t$ as trained on certified i.i.d. positives.
- For intersection-closed classes, after m reliable samples, the generalization error is about $O(d_{\text{vc}}/m)$.
- Summing those mistake probabilities gives a harmonic series:

$$\sum_{t=1}^T O\left(\frac{d_{\text{vc}}}{t}\right) = O(d_{\text{vc}} \log T).$$

- Independently, the hypothesis can move only along a chain of length $\text{ThD}(\mathcal{H})$.



Convex bodies: closure = convex hull (optional application)

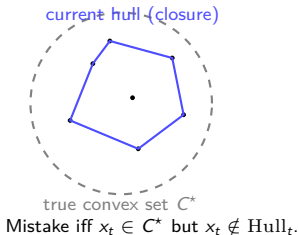
Let $\mathcal{H}_d$ be the class of convex subsets of $\mathbb{R}^d$ (intersection-closed).

Theorem (informal)

For targets supported on a convex set with **nice** bounded density:

$$\mathbb{E}[M_T] = \begin{cases} O(\log T) & d = 1, \\ O\left(T^{\frac{d-1}{d+1}}\right) & d \geq 2, \end{cases}$$

and matching lower bounds hold.



Convex bodies: closure = convex hull (optional application)

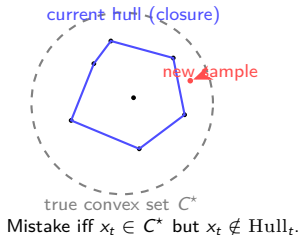
Let $\mathcal{H}_d$ be the class of convex subsets of $\mathbb{R}^d$ (intersection-closed).

Theorem (informal)

For targets supported on a convex set with **nice** bounded density:

$$\mathbb{E}[M_T] = \begin{cases} O(\log T) & d = 1, \\ O\left(T^{\frac{d-1}{d+1}}\right) & d \geq 2, \end{cases}$$

and matching lower bounds hold.



Convex bodies: closure = convex hull (optional application)

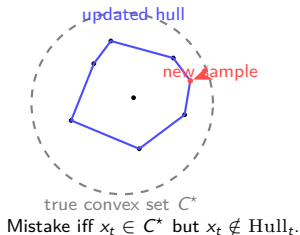
Let $\mathcal{H}_d$ be the class of convex subsets of $\mathbb{R}^d$ (intersection-closed).

Theorem (informal)

For targets supported on a convex set with **nice** bounded density:

$$\mathbb{E}[M_T] = \begin{cases} O(\log T) & d = 1, \\ O\left(T^{\frac{d-1}{d+1}}\right) & d \geq 2, \end{cases}$$

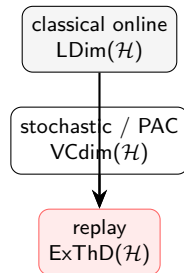
and matching lower bounds hold.



Replay vs. classical online learning

Classical baselines

- **Littlestone dimension** governs online mistake bounds with truthful labels.
- **VC dimension** governs sample complexity and stochastic generalization.



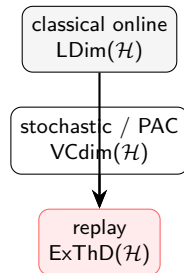
Replay vs. classical online learning

Classical baselines

- **Littlestone dimension** governs online mistake bounds with truthful labels.
- **VC dimension** governs sample complexity and stochastic generalization.

Replay lesson

- Truth is partially hidden, so the classical dimensions alone are not enough.
- Replay introduces a new obstruction: **trap creation**.
- The right adaptive complexity measure is $\text{ExThD}(\mathcal{H})$, not just $\text{LDim}(\mathcal{H})$ or $\text{VCdim}(\mathcal{H})$.



Takeaways

What matters is not just the class, but the feedback loop.

01 Replay changes the learning problem

Past predictions can return as observations, so the learner is no longer seeing clean feedback.

Takeaways

What matters is not just the class, but the feedback loop.

01 Replay changes the learning problem

Past predictions can return as observations, so the learner is no longer seeing clean feedback.

02 Conservative updates are the right response

Update only on certified truth; that prevents replay from manufacturing traps.

Takeaways

What matters is not just the class, but the feedback loop.

01 Replay changes the learning problem

Past predictions can return as observations, so the learner is no longer seeing clean feedback.

02 Conservative updates are the right response

Update only on certified truth; that prevents replay from manufacturing traps.

03 $\text{ExThD}(\mathcal{H})$ is the key quantity

Under adaptive replay, it captures the right mistake complexity.

Questions?

Thank you

Questions?

I am open to collaborations with everyone here, also open to UGP supervisions, and PhD applications.

Language generation: the KM'24 game (standard setting)

Switching gears: replay also matters for **generation** (not just classification).

Protocol

- 1 Adversary secretly picks a target language $h^* \in \mathcal{H}$ (support $\text{supp}(h^*) \subseteq \mathcal{X}$).

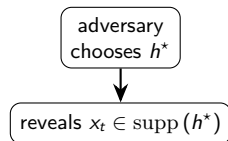
adversary
chooses h^*

Language generation: the KM'24 game (standard setting)

Switching gears: replay also matters for **generation** (not just classification).

Protocol

- 1 Adversary secretly picks a target language $h^* \in \mathcal{H}$ (support $\text{supp}(h^*) \subseteq \mathcal{X}$).
- 2 At round t , adversary reveals a training example $x_t \in \text{supp}(h^*)$.

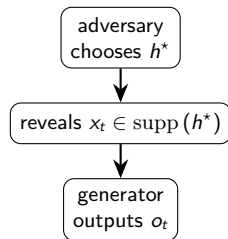


Language generation: the KM'24 game (standard setting)

Switching gears: replay also matters for **generation** (not just classification).

Protocol

- 1 Adversary secretly picks a target language $h^* \in \mathcal{H}$ (support $\text{supp}(h^*) \subseteq \mathcal{X}$).
- 2 At round t , adversary reveals a training example $x_t \in \text{supp}(h^*)$.
- 3 Generator outputs $o_t \in \mathcal{X}$.



Language generation: the KM'24 game (standard setting)

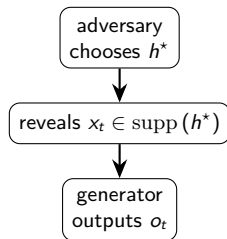
Switching gears: replay also matters for **generation** (not just classification).

Protocol

- 1 Adversary secretly picks a target language $h^* \in \mathcal{H}$ (support $\text{supp}(h^*) \subseteq \mathcal{X}$).
- 2 At round t , adversary reveals a training example $x_t \in \text{supp}(h^*)$.
- 3 Generator outputs $o_t \in \mathcal{X}$.

Success criterion

There exists a burn-in time t^* such that for all $t \geq t^*$, the generator outputs a **valid + novel** token: $o_t \in \text{supp}(h^*) \setminus \{x_1, \dots, x_t\}$.



goal: output
valid & new
tokens after some time

Language generation: the KM'24 game (standard setting)

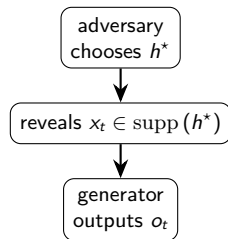
Switching gears: replay also matters for **generation** (not just classification).

Protocol

- 1 Adversary secretly picks a target language $h^* \in \mathcal{H}$ (support $\text{supp}(h^*) \subseteq \mathcal{X}$).
- 2 At round t , adversary reveals a training example $x_t \in \text{supp}(h^*)$.
- 3 Generator outputs $o_t \in \mathcal{X}$.

Success criterion

There exists a burn-in time t^* such that for all $t \geq t^*$, the generator outputs a **valid + novel** token: $o_t \in \text{supp}(h^*) \setminus \{x_1, \dots, x_t\}$.



goal: output
valid & new
tokens after some time

Three settings + baseline

- **In the limit:** t^* may depend on the sample path $(x_t)_{t \geq 1}$.
- **Non-uniform:** t^* may depend on h^* (but not on the sample path).
- **Uniform:** t^* depends only on $\mathcal{H}$.
- **Baseline (KM'24):** countable $\mathcal{H}$ are generatable in the limit; uniform generation has a combinatorial characterization.

Language generation with replay: protocol

Replay protocol (round $t = 1, 2, \dots$)

- 1 Adversary has a target $h^* \in \mathcal{H}$.

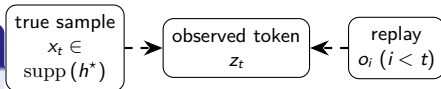
observed token
 z_t

Language generation with replay: protocol

Replay protocol (round $t = 1, 2, \dots$)

- 1 Adversary has a target $h^* \in \mathcal{H}$.
- 2 Adversary reveals an observation z_t where

$$z_t \in \{x_t\} \cup \{o_i : i < t\},$$
$$x_t \in \text{supp}(h^*).$$



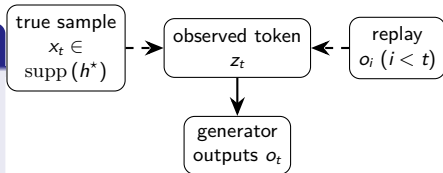
Language generation with replay: protocol

Replay protocol (round $t = 1, 2, \dots$)

- 1 Adversary has a target $h^* \in \mathcal{H}$.
- 2 Adversary reveals an observation z_t where

$$z_t \in \{x_t\} \cup \{o_i : i < t\},$$
$$x_t \in \text{supp}(h^*).$$

- 3 Generator outputs $o_t \in \mathcal{X}$
(cannot tell if z_t was true or replayed).



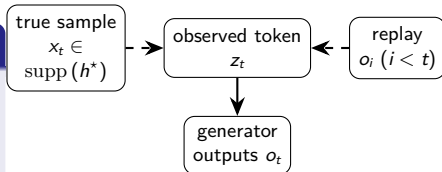
Language generation with replay: protocol

Replay protocol (round $t = 1, 2, \dots$)

- 1 Adversary has a target $h^* \in \mathcal{H}$.
- 2 Adversary reveals an observation z_t where

$$z_t \in \{x_t\} \cup \{o_i : i < t\},$$
$$x_t \in \text{supp}(h^*).$$

- 3 Generator outputs $o_t \in \mathcal{X}$
(cannot tell if z_t was true or replayed).



echo-chamber effect:
bad outputs can
feed back as data

Success criterion (same goal, harder information)

There exists t^* such that for all $t \geq t^*$,

$$o_t \in \text{supp}(h^*) \setminus \{z_1, \dots, z_t\}.$$

Where replay changes what can be generated

Generation notion	Finite $\mathcal{H}$	Countable $\mathcal{H}$	General $\mathcal{H}$
Uniform	✓	✓	✓
Non-uniform (strong)	✓	✗	✗
In the limit	✓	✓	✗

✓: same guarantees as the standard setting. ✗: strict separation. ? : open.

Backup: classification + generation summary

Takeaways

- Replay formalizes *endogenous* contamination (training on past model outputs).
- Core obstruction: a **trap point/region** enables replay loops $\Rightarrow \Omega(T)$ errors.
- Classification: **closure** (conservative, certified-only updates) gives tight bounds via ThD / ExThD.
- Generation: replay yields sharp separations between **uniform** vs. **non-uniform** / **limit** notions.

Open questions

- Tight stochastic replay bounds for **general** $\mathcal{H}$ (beyond intersection-closed).
- Efficiently decide/find an f making $\mathcal{H}^f$ intersection-closed (proper learnability).
- Characterize/compute $\text{ExThD}(\mathcal{H})$ for **infinite** classes beyond special cases (e.g., convex bodies).
- Generation: resolve the remaining separation gaps (e.g. weak non-uniform for general $\mathcal{H}$).

Backup: why closure helps

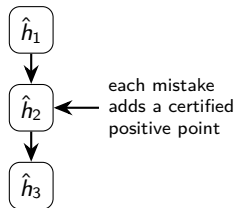
- **Monotone chain:** (as sets) $\hat{h}_1 \subseteq \hat{h}_2 \subseteq \dots$.
- **Certification for free:** if $\hat{h}_t(x_t) = 0$ then all past $\hat{h}_i(x_t) = 0$, so a revealed $y_t = 1$ *cannot be a replay*.
- **Conservative updates:** update only when forced by a certified positive.
- **Trap avoidance:** monotonicity prevents outputting both labels on points that were already made positive.

Lemma 6 (paper, informal)

The closure algorithm only makes mistakes of type *false negative*:

if it errs, then $y_t = 1$ and $\hat{h}_t(x_t) = 0$.

Intuition: it maintains $\hat{h}_t \subseteq f^*$ (in a suitable f -representation), so it never predicts 1 on a true 0.



⋮

mistake
 $y = 1, \hat{y} = 0$

Backup: proof roadmap for Thm. 3

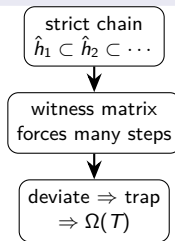
Upper bound: closure

Run closure on $\mathcal{H}^f$ for an f with $\text{ThD}(\mathcal{H}^f) = \text{ExThD}(\mathcal{H})$

- Each mistake adds a **certified positive** x_t with $\hat{h}_t(x_t) = 0$.
- Update strictly increases $\hat{h}_t$ as a set.
- Chain length $\leq \text{ThD}(\mathcal{H}^f) = \text{ExThD}(\mathcal{H})$.

Lower bound: adversary

- If the learner creates a trap, replay loops force $\Omega(T)$ mistakes.
- Therefore any good learner must avoid traps and behave almost monotonically.
- A staircase witness then forces $\Omega(\text{ThD}(\mathcal{H}^f)) = \Omega(\text{ExThD}(\mathcal{H}))$ mistakes.



Backup: reliable version space (certifying truth)

- If y_t disagrees with every past hypothesis on x_t , then y_t **cannot be a replay**.

t	$\hat{h}_{<t}(x_t)$	y_t	Certified ?
1	$\emptyset$	0	Yes
2	$\{0\}$	0	No
3	$\{0\}$	1	Yes
4	$\{0, 1\}$	1	No
5	$\{0, 1\}$	0	No

Backup: reliable version space (certifying truth)

- If y_t disagrees with *every* past hypothesis on x_t , then y_t **cannot be a replay**.
- Define “certified” indices I_t :

$$t \in I_t \iff \forall i < t: \hat{h}_i(x_t) \neq y_t.$$

t	$\hat{h}_{<t}(x_t)$	y_t	Certified ?
1	$\emptyset$	0	Yes
2	$\{0\}$	0	No
3	$\{0\}$	1	Yes
4	$\{0, 1\}$	1	No
5	$\{0, 1\}$	0	No

y_t matches some $\hat{h}_i(x_t)$

$\Rightarrow$ could be replay

y_t matches none
 $\Rightarrow$ must be true

Backup: reliable version space (certifying truth)

- If y_t disagrees with *every* past hypothesis on x_t , then y_t **cannot be a replay**.
- Define “certified” indices I_t :

$$t \in I_t \iff \forall i < t: \hat{h}_i(x_t) \neq y_t.$$

- **Reliable version space** VS_t^* :

$$VS_t^* = \{h \in \mathcal{H} : \forall i \in I_t, h(x_i) = y_i\}.$$

t	$\hat{h}_{<t}(x_t)$	y_t	Certified ?
1	$\emptyset$	0	Yes
2	$\{0\}$	0	No
3	$\{0\}$	1	Yes
4	$\{0, 1\}$	1	No
5	$\{0, 1\}$	0	No

y_t matches some $\hat{h}_i(x_t)$

$\Rightarrow$ could be replay

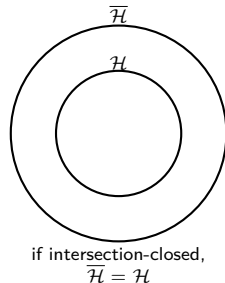
y_t matches none
 $\Rightarrow$ must be true

Interpretation

Only “novel” labels (unexplainable by any past hypothesis) are guaranteed to be from f^* .

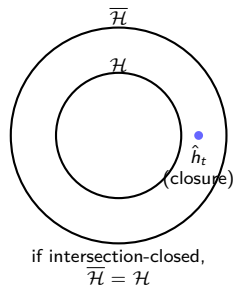
Proper vs. improper: a sharper separation

- Closure algorithm naturally predicts in $\overline{\mathcal{H}}$ (intersection closure).



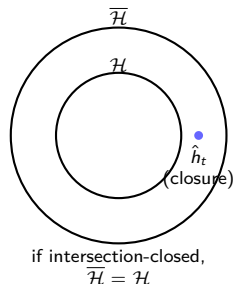
Proper vs. improper: a sharper separation

- Closure algorithm naturally predicts in $\overline{\mathcal{H}}$ (intersection closure).
- If $\mathcal{H}$ is not intersection-closed, closure becomes **improper**.



Proper vs. improper: a sharper separation

- Closure algorithm naturally predicts in $\overline{\mathcal{H}}$ (intersection closure).
- If $\mathcal{H}$ is not intersection-closed, closure becomes **improper**.
- In the replay setting, this is not just a technicality: **proper learning can be impossible**.

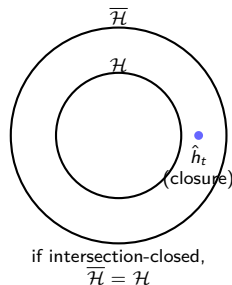


Proper vs. improper: a sharper separation

- Closure algorithm naturally predicts in $\overline{\mathcal{H}}$ (intersection closure).
- If $\mathcal{H}$ is not intersection-closed, closure becomes **improper**.
- In the replay setting, this is not just a technicality: **proper learning can be impossible**.

Theme

Proper learners are forced to “commit” to hypotheses that create trap regions; improper learners can stay conservative inside $\overline{\mathcal{H}}$.



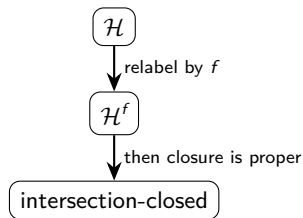
Characterization of proper learnability

Theorem (paper, Thm. 6)

A hypothesis class $\mathcal{H} \subseteq 2^{\mathcal{X}}$ is properly learnable in the replay setting *iff* there exists a representation $f \subseteq \mathcal{X}$ such that the relabeled class

$$\mathcal{H}^f := \{h^f : h \in \mathcal{H}\}$$

is intersection-closed.



Characterization of proper learnability

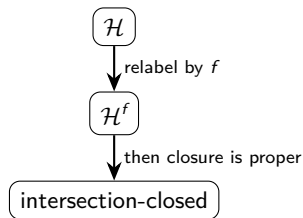
Theorem (paper, Thm. 6)

A hypothesis class $\mathcal{H} \subseteq 2^{\mathcal{X}}$ is properly learnable in the replay setting *iff* there exists a representation $f \subseteq \mathcal{X}$ such that the relabeled class

$$\mathcal{H}^f := \{h^f : h \in \mathcal{H}\}$$

is intersection-closed.

- Sufficiency: run closure algorithm on the intersection-closed representation.
- Necessity: otherwise, any proper learner eventually creates a trap region $\Rightarrow$ unbounded mistakes.



Backup: open problems and extensions (from discussion)

- Characterize $\text{ExThD}(\mathcal{H})$ for **infinite** classes.

Backup: open problems and extensions (from discussion)

- Characterize $\text{ExThD}(\mathcal{H})$ for **infinite** classes.
- Tighten stochastic bounds for **general** classes.

Backup: open problems and extensions (from discussion)

- Characterize $\text{ExThD}(\mathcal{H})$ for **infinite** classes.
- Tighten stochastic bounds for **general** classes.
- Algorithmic question: when does there exist f such that $\mathcal{H}^f$ is intersection-closed?

Backup: open problems and extensions (from discussion)

- Characterize $\text{ExThD}(\mathcal{H})$ for **infinite** classes.
- Tighten stochastic bounds for **general** classes.
- Algorithmic question: when does there exist f such that $\mathcal{H}^f$ is intersection-closed?
- Model extensions: stochastic labels (sample from $\text{Bern}(h(x))$), k -consensus replay, known audited fraction of true labels.

Backup: pointers

- Paper: *Learning in an Echo Chamber: Online Learning with Replay Adversary*, arXiv:2509.25135v1.
- Key notions: reliable version space (Def. 5), closure algorithm (Alg. 1), ExThD (Def. 10), trap region (Def. 11).
- Key results: thresholds (Thm. 1–2), adaptive characterization (Thm. 3), stochastic bounds (Thm. 4–5), proper characterization (Thm. 6).